

Trustworthy Concept-Level Learning Support: Calibrated Exercise-Aware Knowledge Tracing, Unit-Grounded Retrieval and a Closed Remediation Loop

[Author 1 Name]

Dept. of [Computer Science and
Engineering]
[College Name]
[City], India
[email address]

[Author 2 Name]

Dept. of [Computer Science and
Engineering]
[College Name]
[City], India
[email address]

[Guide Name]

Dept. of [Computer Science and
Engineering]
[College Name]
[City], India
[email address]

Abstract—Closed-loop tutoring pipelines that combine knowledge tracing, retrieval-augmented generation and adaptive testing promise individual support at the scale of a whole cohort, but their decisions rest on a per-concept mastery estimate whose quality is rarely examined beyond next-answer accuracy. We build an open, offline-capable pipeline of this kind for outcome-based courses and use it to study three practical questions. First, we show that an exercise-aware knowledge tracing (EKT) model trained only on next-answer prediction produces mastery values that react weakly to new evidence; adding a direct mastery objective leaves the predictive AUC unchanged (0.834) while raising the share of learners whose mastery increases after five correct answers from 74% to 90%. Second, we find that exam exports ordered by concept and difficulty teach both EKT and DKT that mastery falls after correct answers, a defect that AUC does not reveal. Third, splitting the syllabus by unit before chunking raises the share of on-unit retrieved text from 38% to 51%, and restricting retrieval to the unit removes off-topic context entirely. In a simulated cohort of 40 learners, the full loop cleared all flagged gaps for 38 learners after one intervention round, against 27 when re-assessment was used without remediation.

Index Terms—knowledge tracing, calibration, retrieval-augmented generation, remedial recommendation, adaptive assessment, outcome-based education, learning analytics

I. INTRODUCTION

A single examination mark says how much of a course a learner has absorbed, but not which ideas are missing. Outcome-Based Education (OBE) makes the question sharper, because every course outcome carries an explicit threshold and target that the institution has to report against [1], [2]. Answering it for every learner in a large cohort, and then preparing material that addresses each learner's specific weaknesses, is beyond the time a teacher has. This has motivated pipelines that estimate concept mastery automatically, generate remedial content with large language models (LLMs), and re-test the learner, repeating the cycle until the target is met [1].

Such a pipeline acts on its estimates. A concept is marked as a gap, a study plan is produced and a new test is scheduled because a number, the predicted mastery of that concept, is below a cut-off. Knowledge tracing (KT) models are, however,

almost always judged by how well they predict the next answer, typically through the area under the ROC curve (AUC) [3]–[5]. A model can predict the next answer well and still report per-concept mastery that barely moves when the learner starts answering correctly, which is exactly the signal a remediation loop depends on. Two further practical issues arise when such a system is deployed: institutional exam exports often list items grouped by topic and ordered from easy to hard, and fixed-size text chunks used for retrieval can mix material from neighbouring units of a syllabus.

This paper examines these issues with a complete, reproducible implementation. Our contributions are:

- 1) An open, modular closed-loop pipeline for OBE courses that connects curriculum retrieval, EKT, gap detection, LLM-generated and cross-checked remediation, item generation with semantic de-duplication and adaptive re-testing, and that keeps working when commercial model endpoints are unavailable.
- 2) A calibration analysis of the mastery output of EKT, showing that a direct mastery objective is needed for mastery to respond to evidence, at no cost in predictive accuracy.
- 3) Evidence that concept-grouped, difficulty-ordered exam sequences induce a reversed mastery response in both EKT and DKT that standard AUC does not detect, together with a simple responsiveness probe that does.
- 4) A measurement of retrieval purity for three chunking strategies and a cohort-level simulation of the full loop with and without remediation.

Section II reviews related work, Section III describes the system and Section IV its methods. Section V gives the experimental setup, Section VI the results, Section VII discusses implications and threats to validity, and Section VIII concludes.

II. RELATED WORK

A. Knowledge Tracing

Bayesian knowledge tracing treats mastery of each skill as a hidden binary state [6]. Deep knowledge tracing (DKT) replaced it with a recurrent network over interaction sequences [3], followed by memory-based [7], attention-based [4] and deliberately simple [8] variants. Exercise-aware knowledge tracing (EKT) additionally encodes the text of each exercise, so the model knows what was asked and not only which skill it was tagged with [5]. Critiques of DKT have shown that its predictions can be inconsistent and that its skill-level outputs may fluctuate or move against the observed answer, and have proposed regularisers that tie the output to the observed response [9], [10]. Our mastery objective is in the same spirit, applied to EKT and evaluated with an explicit responsiveness probe.

B. Generated Assessment and Retrieval

Retrieval-augmented generation (RAG) conditions an LLM on passages retrieved from a document collection [11], [12], which for education keeps generated material within a prescribed syllabus. LLM-based question generators use retrieval and prompting strategies to produce multiple-choice questions [13]; the pedagogical level of items is commonly expressed with the revised Bloom's taxonomy [14]. Sentence embeddings [15], [16] and approximate nearest-neighbour indexes [17] are the usual building blocks. The effect of chunking on the topical purity of the retrieved context has received little attention in educational settings.

C. Closed-Loop Personalisation

Adaptive e-learning systems and educational recommenders tailor resources to learner profiles [18], [19]. Pradeesh et al. [1] integrated EKT, RAG-based item generation, multi-model verification of generated content and OBE thresholds in a single loop and reported gains in a live course. We follow their overall design and focus on properties that such a loop needs in order to be trustworthy: the behaviour of the mastery signal, sensitivity to the ordering of exported data, retrieval purity and robustness to unavailable model services.

III. SYSTEM OVERVIEW

Fig. 1 shows the pipeline. A teacher uploads the curriculum once; it is split by unit, embedded and indexed, and one knowledge concept is proposed per unit for the teacher to approve. Learner data arrive as OBE records, one row per answered item with the concept, difficulty, maximum marks and marks obtained. For each learner, EKT estimates mastery per concept; concepts below the OBE target are flagged, a remediation plan grounded in the concept's own unit is generated and verified, fresh items are produced for the flagged concepts, and an adaptive test re-assesses the learner. The responses are appended to the learner's history and the cycle repeats.

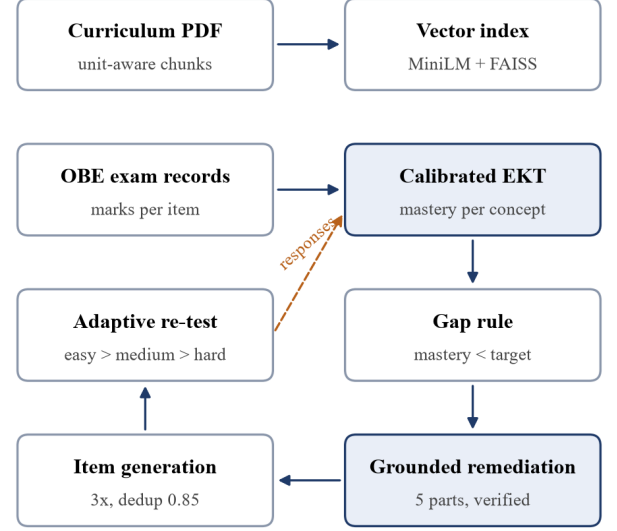


Fig. 1. Closed-loop pipeline. Remediation and item generation retrieve text only from the flagged concept's own curriculum unit; responses to the adaptive re-test are fed back to knowledge tracing.

The implementation is a Python package with a command-line interface, a REST API and a single-page web client. All thresholds and hyper-parameters are kept in one configuration file, and every experiment in this paper can be regenerated from the command line.

IV. METHODS

A. Unit-Grounded Curriculum Retrieval

The curriculum text is divided at unit headings. Each unit is cut into chunks of about 1000 characters with 100 characters of overlap, where cuts are moved to the nearest sentence end and each overlap begins at a sentence start, so that no chunk starts or ends inside a word. Chunks are embedded with all-MiniLM-L6-v2 [16] and stored in an inner-product index over normalised vectors [17]. To build context for concept c , the query is the concept name with its learning objectives; chunks of c 's own unit are ranked by cosine similarity and only those are used. We quantify the benefit with the purity of a retrieved context R for unit u ,

$$purity(R, u) = |\{s \sqsubseteq sent(R) : s \sqsubseteq u\}| / |sent(R)| \quad (1)$$

where $sent(R)$ is the multiset of sentences in the retrieved chunks. Coverage is the share of u 's sentences that appear in R .

B. EKT with a Calibrated Mastery Output

For interaction t on concept c_t with outcome r_t (1 if the marks reach the OBE threshold), the exercise text is embedded as x_t and combined with a learned concept embedding:

$$e_t = \tanh(W_e x_t + E[c_t]) \quad (2)$$

The input to an LSTM [20] places e_t in one of two halves depending on the outcome, $u_t = [e_t, 0]$ when $r_t = 1$ and $[0, e_t]$

otherwise, giving the state $h_t = \text{LSTM}(u_t, h_{t-1})$. A linear layer with a sigmoid produces the mastery vector used for all decisions,

$$m_t = \sigma(W_m h_t + b_m) \quad (3)$$

and the next outcome is predicted from the mastery of the next concept plus an exercise-aware term, with h'_t an attention-weighted sum of earlier states whose weights are the softmax of the cosine similarity between e_{t+1} and each earlier exercise [5]:

$$p_{t+1} = \sigma(\text{logit } m_t[c_{t+1}] + g([h'_t; e_{t+1}])) \quad (4)$$

Trained only on the next-answer loss, the mastery head m_t receives gradient solely through the sum inside the sigmoid, and the exercise-aware term g can absorb most of the signal. We therefore add a direct mastery objective and minimise

$$L = \text{BCE}(p_{t+1}, r_{t+1}) + \lambda \text{BCE}(m_t[c_{t+1}], r_{t+1}) \quad (5)$$

with $\lambda = 0.5$. DKT [3] serves as the baseline; it receives a one-hot encoding of (concept, outcome) and outputs the mastery vector directly.

C. Gap Detection

With the OBE threshold $\theta = 50\%$ and target $\tau = 70\%$, a concept is at the Beginner level when $m < \theta$, Intermediate when $\theta \leq m < \tau$ and Expert when $m \geq \tau$. Every concept with $m < \tau$ is a gap. The decision uses the traced mastery rather than the raw exam score, so a single lucky or unlucky item carries limited weight.

D. Grounded Remediation with Cross-Model Verification

For each gap, the primary LLM receives the unit's retrieved text, the learner's current mastery and the items attempted in the latest session, and returns a plan with five parts: learning objectives, topics to revise, a remedial explanation, two practice activities and a rationale naming the detected weakness. Two further models score the plan from 1 to 5; if the mean is below 3 the plan is regenerated with their comments, at most twice. Later rounds ask for more detailed explanations and simpler examples.

E. Item Generation and De-duplication

For a concept and level, the generator asks for three times the required number of four-option items at the corresponding Bloom levels (easy: remember, understand; medium: apply, analyse; hard: evaluate, create) [14]. Each item is refined by the same model, then compared with every item already stored: an item whose question and key have a cosine similarity above 0.85 to any stored item is discarded. The remaining items are scored by the verifier models and kept when the mean score is at least 3.

F. Adaptive Re-assessment

A rule written as "n and p" asks n items per level and requires p percent correct to advance, for example "5 and 80". Learners start at the easy level of every flagged concept and see a harder level only after passing the current one; items already shown to the learner are excluded. The highest level passed gives the category (none: not competent; easy: beginner; medium: intermediate; hard: expert), and every response is stored as a new OBE record.

G. Fault-Tolerant Model Access

All generative calls share one interface returning structured JSON. Each commercial provider is wrapped so that a failed call is answered by a deterministic offline generator with the same output schema. Account-level errors, such as missing credit, an invalid key or a retired model, disable that provider for the rest of the run, so a failing service costs one round-trip instead of one per request.

V. EXPERIMENTAL SETUP

A. Data

No institutional records were available, so learner data were simulated with a two-parameter-logistic item response model [21]. Each simulated learner has an ability θ_k per concept, drawn from a general ability, a concept offset and individual noise; the probability of attaining an item of difficulty b is

$$P(r = 1) = 1 / (1 + \exp(-1.7 (\theta_k - b))) \quad (6)$$

with $b = -1, 0$ and 1 for easy, medium and hard items and a small practice increment after each answer. The course is "Procedural Programming using C" with five units and 30 items. The training set has 600 learners with two 20-item exams each (24,000 records); a separate cohort of 262 learners with nine answers each is used for the loop. For the ordering study the same learners and random draws were used, but each exam presented its items grouped by concept and sorted from easy to hard.

B. Measures

Next-answer prediction is measured by AUC, accuracy and F1 on held-out learners (75/25 split). Mastery quality is measured in two ways. The mastery-output AUC uses $m_t[c_{t+1}]$ alone as the predictor of r_{t+1} . The responsiveness probe appends five additional records for one concept to a held-out learner's history, all attained or all failed, and records the change in that concept's mastery; it is repeated for every concept of 120 held-out learners. A trustworthy mastery signal should rise after correct answers and fall after wrong ones.

C. Implementation

Both KT models use a 100-dimensional embedding, a 100-unit LSTM, dropout 0.3, the Adam optimiser [22] with learning rate 0.001, batches of 32 and 12 epochs. Each configuration was trained with three seeds (42, 7, 123) and we report mean and standard deviation. Experiments ran on a laptop CPU (Intel Core i5-1335U, 16 GB RAM) with PyTorch [23]. The commercial endpoints configured for generation and verification were not usable during the study (no remaining credit on two accounts and a retired model name on the third), so all generation reported here used the offline generator. Question-quality results are therefore outside the scope of this paper.

VI. RESULTS

A. Predictive Accuracy

TABLE I
NEXT-ANSWER PREDICTION ON HELD-OUT LEARNERS (MEAN $\pm$ S.D., 3 SEEDS)

Model	AUC	Accuracy	F1
DKT	0.683 $\pm$ 0.016	0.644 $\pm$ 0.014	0.712 $\pm$ 0.016
EKT (full)	0.834 $\pm$ 0.006	0.759 $\pm$ 0.009	0.795 $\pm$ 0.011
EKT w/o attention	0.829 $\pm$ 0.007	0.754 $\pm$ 0.007	0.791 $\pm$ 0.004
EKT w/o mastery loss	0.834 $\pm$ 0.006	0.758 $\pm$ 0.009	0.796 $\pm$ 0.007
EKT + exercise dropout	0.831 $\pm$ 0.006	0.757 $\pm$ 0.010	0.791 $\pm$ 0.008

Table I shows that reading the exercise text pays off: EKT reaches an AUC of 0.834 ± 0.006 against 0.683 ± 0.016 for DKT, with a matching gain in F1. Removing the mastery objective leaves prediction unchanged (0.834), and removing attention costs only 0.005. Judged by AUC alone, the three EKT variants are interchangeable.

B. Calibration of the Mastery Output

TABLE II
MASTERY OUTPUT: AUC AND RESPONSE TO FIVE FURTHER ANSWERS

Model	Mastery AUC	Δ after 5 correct	Rises	Δ after 5 wrong	Falls
DKT	0.683	+6.9 $\pm$ 0.7	98%	-8.7 $\pm$ 1.3	100%
EKT (full)	0.693	+9.9 $\pm$ 0.6	90%	-11.5 $\pm$ 2.7	94%
EKT w/o attention	0.696	+13.5 $\pm$ 1.0	97%	-16.5 $\pm$ 1.6	97%
EKT w/o mastery loss	0.661	+5.7 $\pm$ 0.6	74%	-6.7 $\pm$ 2.0	79%
EKT + exercise dropout	0.690	+9.3 $\pm$ 1.8	91%	-8.3 $\pm$ 2.1	88%

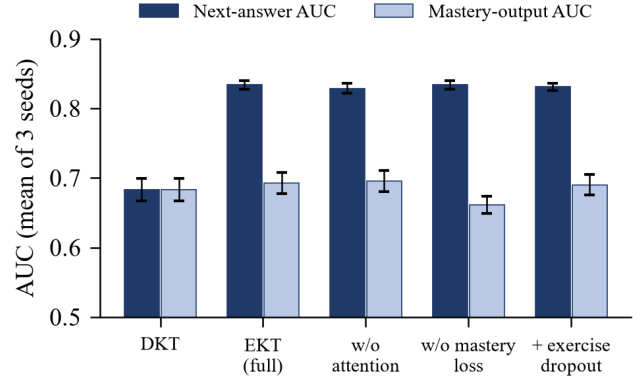


Fig. 2. Next-answer AUC and mastery-output AUC of the knowledge-tracing variants. Error bars show the standard deviation over three seeds.

The picture changes when the mastery output itself is examined (Table II, Fig. 2). Without the mastery objective, EKT's mastery AUC drops from 0.693 to 0.661, and after five correct answers mastery rises in only 74% of the probes, by 5.7 points on average, against 90% and 9.9 points with it. The response to wrong answers shows the same pattern (79% against 94% falling). A loop driven by the uncalibrated head would therefore keep flagging concepts the learner has already recovered.

Attention trades responsiveness for a small gain in prediction: without it, mastery rises in 97% of the probes by 13.5 points, because the prediction no longer relies on similar earlier states. DKT, whose output is the mastery vector, responds in the expected direction almost always but with smaller steps (6.9 points), and its predictive AUC is far lower. Hiding the exercise text half of the time during training (exercise dropout) did not improve either measure.

C. An Ordering Artifact Invisible to AUC

TABLE III
EFFECT OF EXAM ITEM ORDER ON TRAINING

Model	Exam order	AUC	Δ after 5 correct	Rises
EKT	shuffled	0.834	+9.9 $\pm$ 0.6	90%
EKT	ordered	0.807	-1.4 $\pm$ 1.3	50%
DKT	shuffled	0.683	+6.9 $\pm$ 0.7	98%
DKT	ordered	0.765	-2.5 $\pm$ 2.6	50%

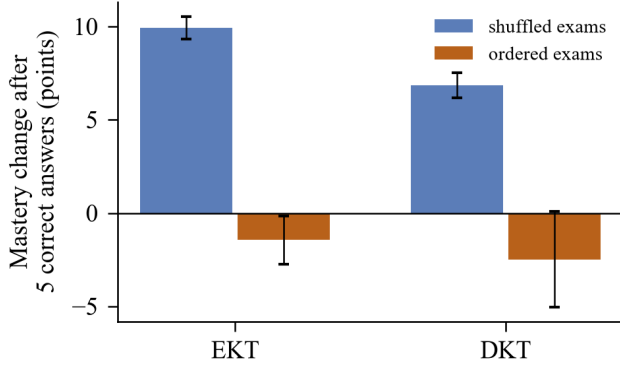


Fig. 3. Mean change in concept mastery after five additional correct answers, for models trained on shuffled and on concept-grouped, easy-to-hard exams.

When exams list items grouped by concept and sorted by difficulty, both models learn that a run of answers on one concept is followed by a harder item and a likely failure. After five correct answers their mastery falls on average (-1.4 points for EKT, -2.5 for DKT) and rises in only about half of the probes (Table III, Fig. 3), even though the predictive AUC stays respectable (0.807 and 0.765). The mastery-output AUC is even higher on ordered data (0.777), because position within the exam is itself predictive. We first observed this defect in our own pipeline as mastery values that decreased after a learner answered nine questions correctly; standard evaluation had not revealed it.

D. Retrieval Purity

TABLE IV

SHARE OF RETRIEVED TEXT FROM THE TARGET UNIT (TOP-4 CHUNKS)

Strategy	Purity, 500	Purity, 1000	Coverage, 1000
Fixed windows	63.7%	38.4%	98.5%
Unit-aware chunks	68.8%	50.9%	100.0%
Unit-aware + unit filter	100.0%	100.0%	100.0%

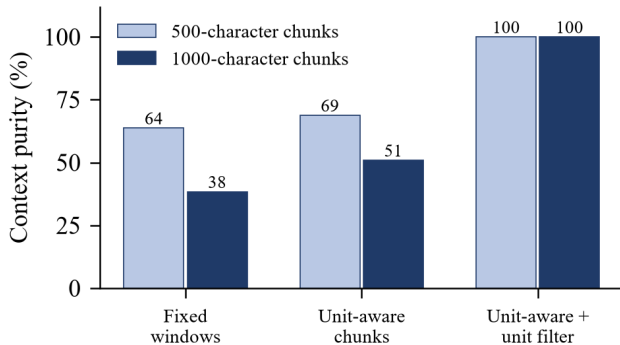


Fig. 4. Context purity of three chunking and retrieval strategies for two chunk sizes.

With 1000-character windows over the whole document, only 38.4% of the retrieved sentences belonged to the concept's unit (Table IV, Fig. 4); the rest came from

neighbouring units, so a question about strings could be built on a sentence about pointers, which we observed in early runs. Splitting by unit before chunking raised purity to 50.9%, and restricting retrieval to the unit made every retrieved sentence on-topic by construction while still covering the whole unit. Smaller chunks are purer (63.7% for fixed 500-character windows) but cover less of the unit (72.4%).

E. Closed-Loop Simulation

TABLE V

COHORT SIMULATION: LEARNERS WITH NO OPEN GAP, AND MEAN OPEN GAPS PER ROUND

Condition	After 1 cycle	After 3 cycles	R1	R2	R3	R4
With remediation	38 / 40	40 / 40	4.50	0.05	0.03	0.00
Without remediation	27 / 40	35 / 40	4.50	0.85	0.47	0.28

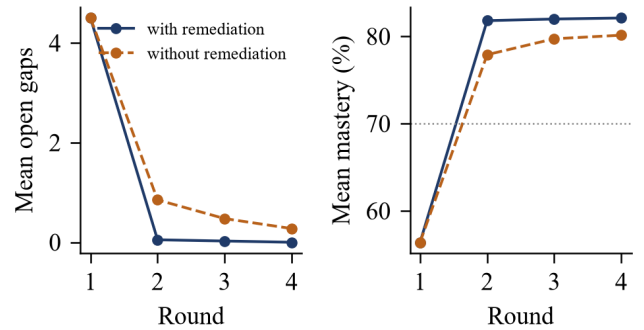


Fig. 5. Mean number of open gaps and mean mastery per round for 40 simulated learners, with and without the remediation step. The dotted line marks the 70% target.

We ran the full loop for 40 learners of the second cohort, with the rule "3 and 67" and at most three cycles. Learners answered the adaptive tests according to their simulated ability; in the remediation condition, studying a plan raised the ability on that concept by a random amount (mean 0.8 logits). All learners started with 4.5 flagged concepts on average, reflecting how little evidence nine exam answers provide. With remediation, 38 learners had no open gap after one cycle and all 40 after three. Without remediation, re-assessment alone resolved many gaps, because the extra answers corrected underestimated mastery, but 5 learners still had open gaps after three cycles (Table V, Fig. 5). The difference between the conditions follows from the simulated learning gain and should be read as a check that the loop reacts correctly to learning, not as an estimate of real learning gains.

F. Robustness

During the study, each of the three configured commercial providers failed with an account-level error. With the fault-tolerant wrapper, each provider was contacted once per run, and every later request was served by the offline generator. Before this change, every request waited for a failing network call, which made a full experimental run impractically slow.

VII. DISCUSSION

The results suggest three practical rules for concept-level tutoring systems. First, a knowledge-tracing model that drives decisions should be evaluated on the quantity those decisions use; a responsiveness probe such as ours takes minutes to run and exposed defects that predictive AUC did not. Second, the ordering of exported interaction data matters: records should carry timestamps or at least a randomised within-exam order, and models should be checked on shuffled data before deployment. Third, retrieval for concept-specific generation should respect the structure of the syllabus rather than fixed character windows.

Threats to validity. All learner data are simulated, and the simulator's assumptions (a single ability per concept, logistic responses and a fixed practice effect) shape the results; effects on real learners may be smaller or larger. The responsiveness probe re-uses the course's own items and a fixed number of added answers. The curriculum has five units, so the retrieval measurements cover a small document. Generated remediation and item quality were not evaluated, because the commercial models were unavailable. Finally, the closed-loop comparison depends on the simulated effect of studying a plan.

VIII. CONCLUSION

We built an open, fault-tolerant pipeline that traces concept mastery, detects gaps against OBE targets, generates grounded and cross-checked remediation, and re-assesses learners adaptively. Using it, we showed that the mastery output of EKT needs a direct objective to respond to new evidence, that ordered exam exports can reverse the response of both EKT and DKT without lowering AUC noticeably, and that unit-aware retrieval removes off-topic context. Future work will evaluate the pipeline with real OBE records and pre- and post-tests, measure the quality of LLM-generated items and plans with human raters, and extend the calibration analysis to further knowledge-tracing architectures [4], [7], [8].

ACKNOWLEDGMENT

The authors thank [Guide Name] and the Department of [Computer Science and Engineering], [College Name], for their guidance and support. The system design follows the framework of Pradeesh et al. [1].

REFERENCES

- [1] N. Pradeesh, M. G. Thushara, K. Arun Krishna, V. Pranav, and S. Krishnamoorthy, "AI-driven personalized learning and remedial recommendation through knowledge concept-centric evaluation," *IEEE Access*, vol. 13, pp. 207817-207837, 2025.
- [2] R. M. Harden, "Developments in outcome-based education," *Medical Teacher*, vol. 24, no. 2, pp. 117-120, 2002.
- [3] C. Piech et al., "Deep knowledge tracing," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015.
- [4] A. Ghosh, N. Heffernan, and A. S. Lan, "Context-aware attentive knowledge tracing," in *Proc. ACM SIGKDD*, 2020, pp. 2330-2339.
- [5] Q. Liu et al., "EKT: Exercise-aware knowledge tracing for student performance prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 100-115, 2021.

- [6] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Model. User-Adapt. Interact.*, vol. 4, no. 4, pp. 253-278, 1994.
- [7] J. Zhang, X. Shi, I. King, and D.-Y. Yeung, "Dynamic key-value memory networks for knowledge tracing," in *Proc. Int. Conf. World Wide Web (WWW)*, 2017.
- [8] Z. Liu, Q. Liu, J. Chen, S. Huang, and W. Luo, "simpleKT: A simple but tough-to-beat baseline for knowledge tracing," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [9] C.-K. Yeung and D.-Y. Yeung, "Addressing two problems in deep knowledge tracing via prediction-consistent regularization," in *Proc. ACM Conf. Learning at Scale*, 2018.
- [10] M. Khajah, R. V. Lindsey, and M. C. Mozer, "How deep is knowledge tracing?" in *Proc. Int. Conf. Educational Data Mining (EDM)*, 2016.
- [11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459-9474.
- [12] W. Fan et al., "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in *Proc. ACM SIGKDD*, 2024, pp. 6491-6501.
- [13] C. N. Hang, C. W. Tan, and P.-D. Yu, "MCQGen: A large language model-driven MCQ generator for personalized learning," *IEEE Access*, vol. 12, pp. 102261-102273, 2024.
- [14] L. W. Anderson and D. R. Krathwohl, Eds., *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York, NY, USA: Longman, 2001.
- [15] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019.
- [16] W. Wang et al., "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. NeurIPS*, 2020.
- [17] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [18] M. Murtaza, Y. Ahmed, J. A. Shamsi, F. Sherwani, and M. Usman, "AI-based personalized e-learning systems: Issues, challenges, and solutions," *IEEE Access*, vol. 10, pp. 81323-81342, 2022.
- [19] N. S. Raj and V. G. Renumol, "A systematic literature review on adaptive content recommenders in personalized learning environments from 2015 to 2020," *J. Comput. Educ.*, vol. 9, no. 1, pp. 113-148, 2022.
- [20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [21] F. M. Lord, *Applications of Item Response Theory to Practical Testing Problems*. Hillsdale, NJ, USA: Lawrence Erlbaum, 1980.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [23] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.